eProject Comprehensive Report

EventSphere – Event & Expo Management System

Group Project · Batch 202410C1

Submitted by

Sufiyan | 1617416

Haris Gaba | 1619067

Munaem Javed | 1616152

Project Supervisor

Sir Arsalan Warsi

Submission Date

20 November 2026

React, Vite, Tailwind CSS, Express, Node.js and MongoDB

Student Team Profiles and Division of Work

The EventSphere eProject was completed by three students in batch 202410C1 under the supervision of Sir Arsalan Warsi.

No.	Student ID	Student name	Primary contribution
1	1617416	Sufiyan	Frontend Development and UI/UX Design
2	1619067	Haris Gaba	Backend Development and API Integration
3	1616152	Munaem Javed	Database Management and Testing

Individual Contribution Matrix

Sufiyan was responsible for frontend development and user interface and experience design.

Haris Gaba was responsible for backend development and integration between the frontend and the API.

Munaem Javed was responsible for database management and testing.

Jury Evaluation Sheet

The jury may enter individual scores after reviewing the implementation and conducting the project defense.

Student	Frontend / 25	Backend / 25	Database / 25	Viva / 25	Total / 100
Sufiyan					
Haris Gaba					
Munaem Javed					

Jury Panel and Official Verdict

Evaluator name and signature

_____

Overall verdict

_____

Remarks

_____

Table of Contents

Student Team Profiles and Division of Work 2
Jury Evaluation Sheet 3
1. Acknowledgements 5
2. eProject Synopsis 5
3. eProject Analysis 6
3.2 Functional Requirements 7
3.3 Non-Functional Requirements 8
4. eProject Design 9
4.1 Data Flow Diagrams 9
4.1.2 DFD Level 1 10
4.1.3 DFD Level 2 11
4.2 System Flowchart 12
4.3 UML Sequence Diagram 13
4.4 Database Design 14
4.4.1 Operational Data Dictionary 15
4.4.2 Supporting Data Dictionary 16
5. Package Dependencies and Technical Justification 17
6. System Folder Structure 20
7. Screenshots and Interface Walkthrough 21
8. Source Code with Comments 27
8.3 Transactional Booth Reservation 28
8.4 Idempotent Booking Check-In 29
9. User Guide 30
9.3 Administrator and Organizer Workflows 31
9.4 Exhibitor Workflow 32
10. Developer Guide and Module Descriptions 33
10.3 API Route Reference 34
10.4 Verification and Troubleshooting 37

1. Acknowledgements

We thank our project supervisor, Sir Arsalan Warsi, for guiding the EventSphere eProject. We also acknowledge the work of each team member in developing the interface, integrating the application services, managing the database and testing the system.

2. eProject Synopsis

2.1 Executive Summary

EventSphere is a web application for publishing Events and Expos, coordinating exhibitor participation and managing attendee bookings. A public website presents published listings and event information. Authenticated workspaces support administration, organization, exhibition participation and attendance.

2.2 Industry Background and Problem Statement

When listings, booth assignments and attendee approvals are tracked in separate documents, staff must reconcile decisions manually. A stale availability view can lead two exhibitors to request the same booth. Entry staff also need a dependable way to verify that a presented pass belongs to a confirmed booking and to record whether it has already been used.

2.3 Proposed Solution and Scope

EventSphere maintains listings, applications, booths, bookings and sessions in a shared MongoDB database. It applies role and ownership checks to administrative operations. Approved bookings can produce signed QR passes, and eligible staff can record attendance on the event date. Expo operations also include floor plans, exhibitor files and staff details, service requests, and booth visit records.

2.4 Project Objectives

- Make approved listings discoverable through the public site while retaining a role-scoped review workspace.
- Use database uniqueness and conditional updates to prevent duplicate bookings and conflicting booth assignments.
- Track attendee approval, pass issuance and event-day check-in against a single booking record.
- Provide session activity, communications and operational summaries within the same application.

3. eProject Analysis

3.1 Feasibility Study

Technical feasibility. The browser application uses React and Vite. Express exposes HTTP endpoints, and Mongoose models define storage and validation. MongoDB transactions allow related booth writes to commit together. A replica set or compatible managed deployment is required for those operations.

Operational feasibility. Visitors can browse listings before signing in. Staff and participants work through role-specific navigation. Explicit review and booking states give staff a consistent way to determine which requests still need action.

Economic feasibility. The development stack can run on a single workstation using locally installed tools and a test database. Deployment costs depend on the chosen web host, database service and mail account.

Scope and System Boundaries

Area	Implementation boundary
Public site	Landing page, Events and Expos, listing details, gallery, services, contact email composition, public reviews and pass verification.
Workspace	Role-scoped dashboards, listing review, applications, bookings, booths, floor plans, schedules, messages, feedback, notifications, analytics and administration.
External services	SMTP transports password recovery emails. A public HTTPS frontend origin is required for QR verification links.
Storage	MongoDB stores operational documents, selected uploaded media and the website logo. Browser storage retains the login state and website banner settings for that browser.

Operational comparison

A shared listing record lets visitors discover approved information while reviewers handle pending work in the workspace. Reservations and check-ins operate on database records so staff can query current status and the server can enforce conflicts at the point of update.

3.2 Functional Requirements

ID and actors	Implemented behavior
FR-01 · Visitor	Browse published Events and Expos, read public reviews and look up public expo booths, floor plans, sessions or exhibitor profiles through their respective endpoints.
FR-02 · Account holder	Register as an attendee or exhibitor, sign in and request a password reset link. Admin and organizer accounts require existing authorized provisioning.
FR-03 · Admin, organizer, exhibitor	Create listings according to role. New listings start Pending. Admin reviews Expos. The assigned exhibitor reviews regular Events. Organizers manage their own listings.
FR-04 · Attendee and reviewers	Create one booking per listing and start it Pending. Admin reviews Expo bookings. The assigned exhibitor reviews regular Event bookings.
FR-05 · Attendee and entry staff	Retrieve a confirmed booking pass. Public verification reports its validity. An authorized Admin or assigned exhibitor can record entry on the event date in Pakistan time.
FR-06 · Exhibitor and operations staff	Submit and review expo applications, manage company details, staff and files, request operational services, and reserve, cancel or assign booths subject to role and ownership.
FR-07 · Admin, organizer, attendee	Maintain an Expo floor plan. Attendees can consult booth placement and log eligible booth visits.
FR-08 · Admin, organizer, attendee	Manage sessions, browse schedules, and record attendee registration or bookmark activity. Eligible users receive in-app session reminders.
FR-09 · Signed-in user and staff	Open permitted event message threads, submit feedback, read notifications, and handle feedback or service requests under staff permissions.
FR-10 · Admin and organizer	View operational analytics. Admin can list user accounts and manage the shared site logo.

Client navigation limits which workspace screens each role can open. Express middleware and individual controllers enforce permissions and ownership for the underlying actions.

3.3 Non-Functional Requirements

Quality attribute	Implemented control and evaluation
Access control	JWT bearer tokens expire after one hour. Middleware verifies signature, issuer, audience, subject and token version. Route and resource checks enforce role and ownership.
Account protection	bcrypt uses cost 12. Password recovery stores a SHA-256 hash of a random token with a 15-minute expiry. Successful reset consumes the token and increments tokenVersion.
Data consistency	Compound unique indexes prevent repeated bookings, applications and booth identifiers. Booth allocation groups related writes in a transaction. Listing review checks the submitted revision timestamp.
Input validation	Server-side field and identifier checks complement the frontend forms. JSON bodies are limited to 100 KB. Upload routes apply their own file limits and validation.
Responsiveness	Vite splits lazy-loaded routes into chunks. Axios has a 15-second request timeout. Selected updates trigger client refresh through server-sent events.
Usability	The public site and workspace use responsive layouts and provide feedback for loading, empty and failed operations.
Maintainability	Routes, controllers, models, services and UI components have separate source directories. Existing automated tests cover focused business rules.

3.4 Hardware and Software Environment

A development workstation needs a supported Node.js runtime, npm, Git, a modern browser and access to MongoDB. The frontend build and backend test run used Node.js 24.22.1. The local frontend defaults to port 5173 and the API to port 3000. QR scanning depends on a browser and device capable of camera access and barcode detection, or the manual entry option in the scanner.

The project defines application safeguards and observable behaviors. Quantitative capacity and uptime targets require deployment measurements.

4. eProject Design

4.1 Data Flow Diagrams

Level 0 establishes the boundary of the web platform. The public visitor discovers published information. Signed-in roles provide requests and receive decisions or operational data. The SMTP service receives password recovery messages for delivery.

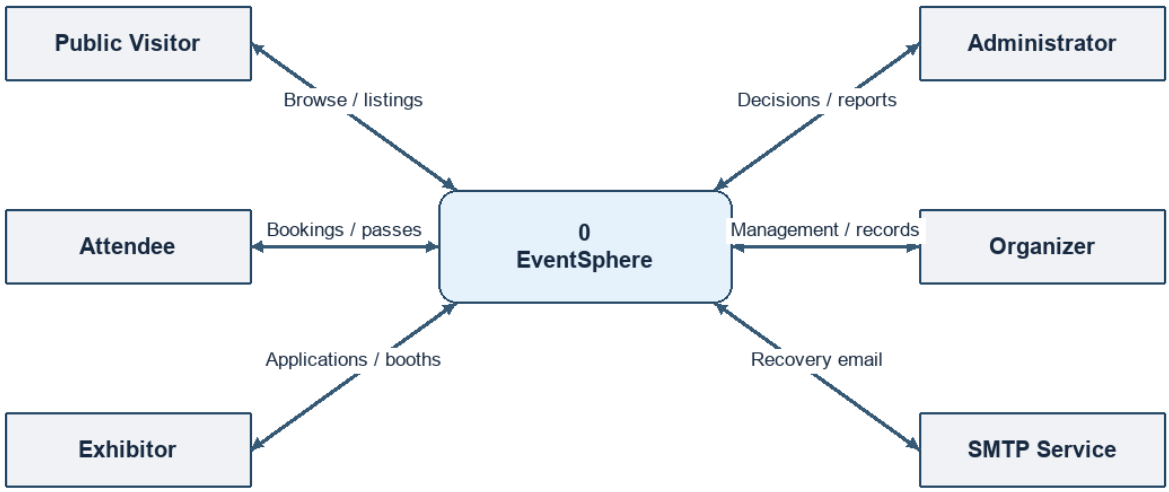


Figure 4.1. DFD Level 0 showing the EventSphere system context.

Actor and system boundary

An account can use public pages without workspace access. The Express API validates the identity on protected operations. The SMTP integration serves the password recovery workflow.

4.1.2 DFD Level 1

Level 1 shows the main processes and the stores they access. The grouped stores identify related collections. A two-headed arrow represents requests and responses or reads and writes. Analytics reads operational data.

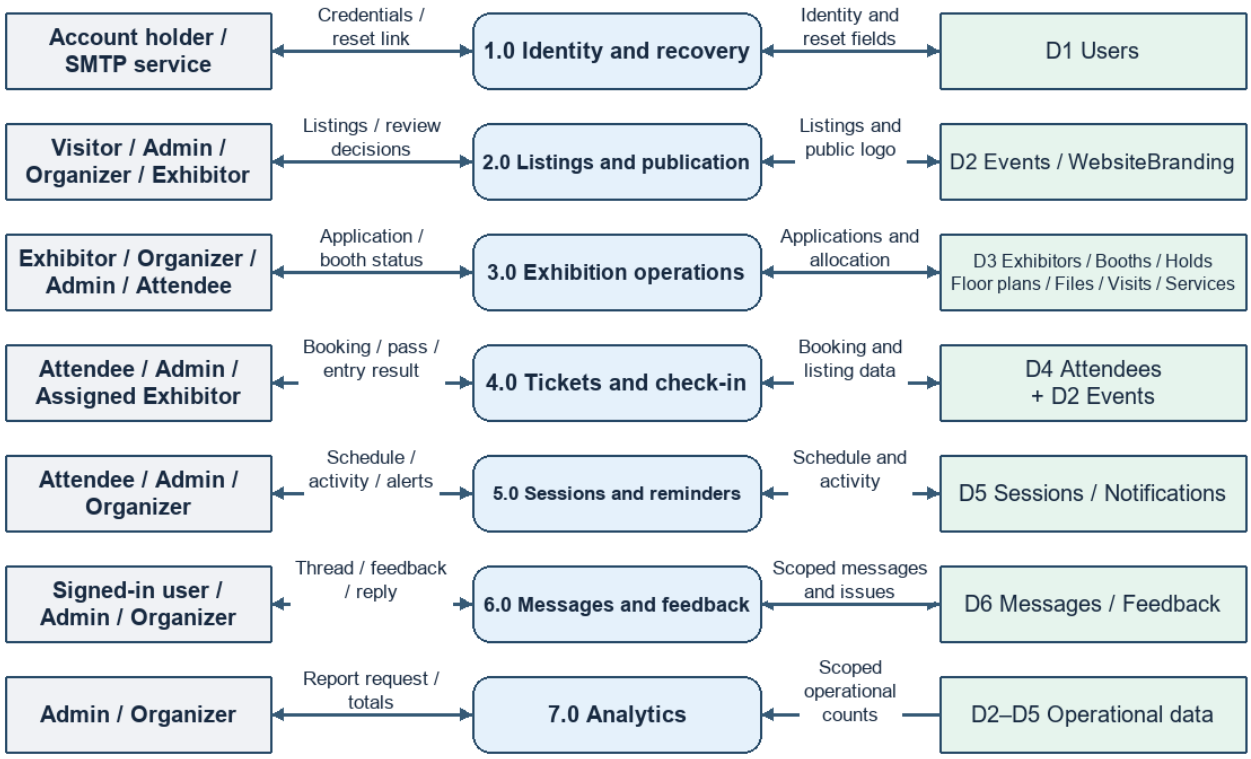


Figure 4.2. DFD Level 1 showing process decomposition and data stores.

Identity checks apply to protected calls across the diagram. The communications process stores feedback and event threads, while the session process creates reminder notifications. The ticket process reads listing information before accepting or confirming a booking.

4.1.3 DFD Level 2

The reservation path decomposes process 3.0. An exhibitor supplies a booth ID. The controller resolves its listing and requires an approved application associated with the signed-in user.

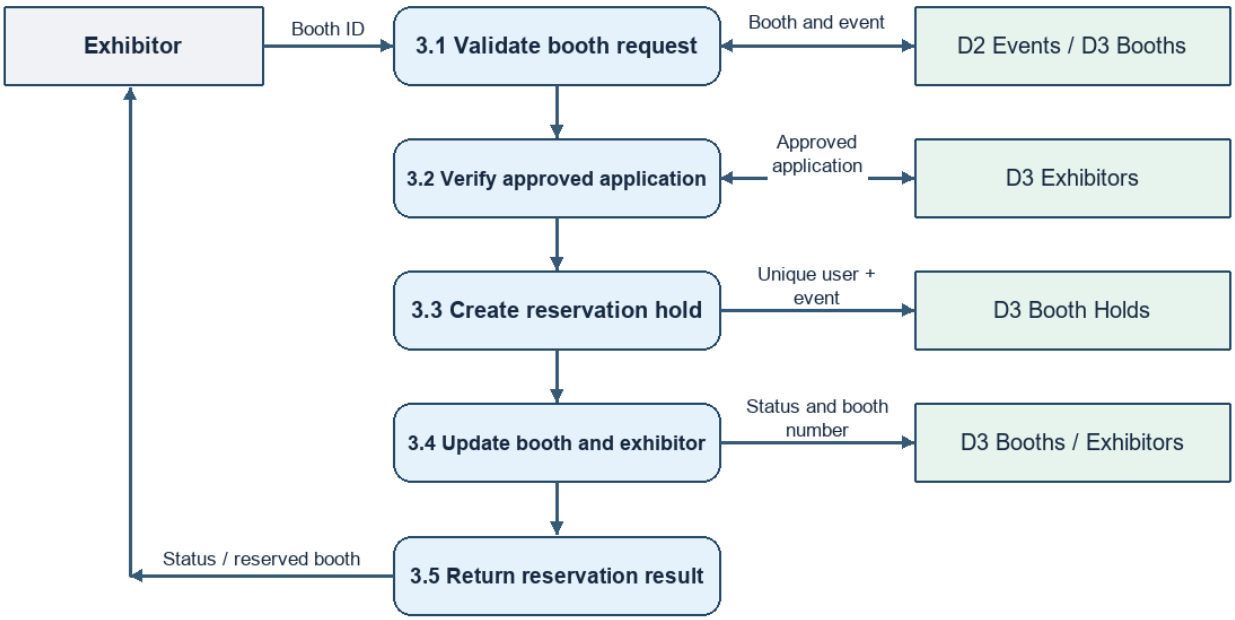


Figure 4.3. DFD Level 2 showing exhibitor self-service booth reservation.

The transaction writes a unique user-and-event hold, conditionally reserves an available booth, and saves the booth number on the exhibitor record. If validation fails or the booth has been taken, the API returns an error and the transaction does not commit.

4.2 System Flowchart

The flowchart follows a booking from the public listing to the entry desk. The reviewer is the administrator for an Expo or the assigned exhibitor for a regular Event.

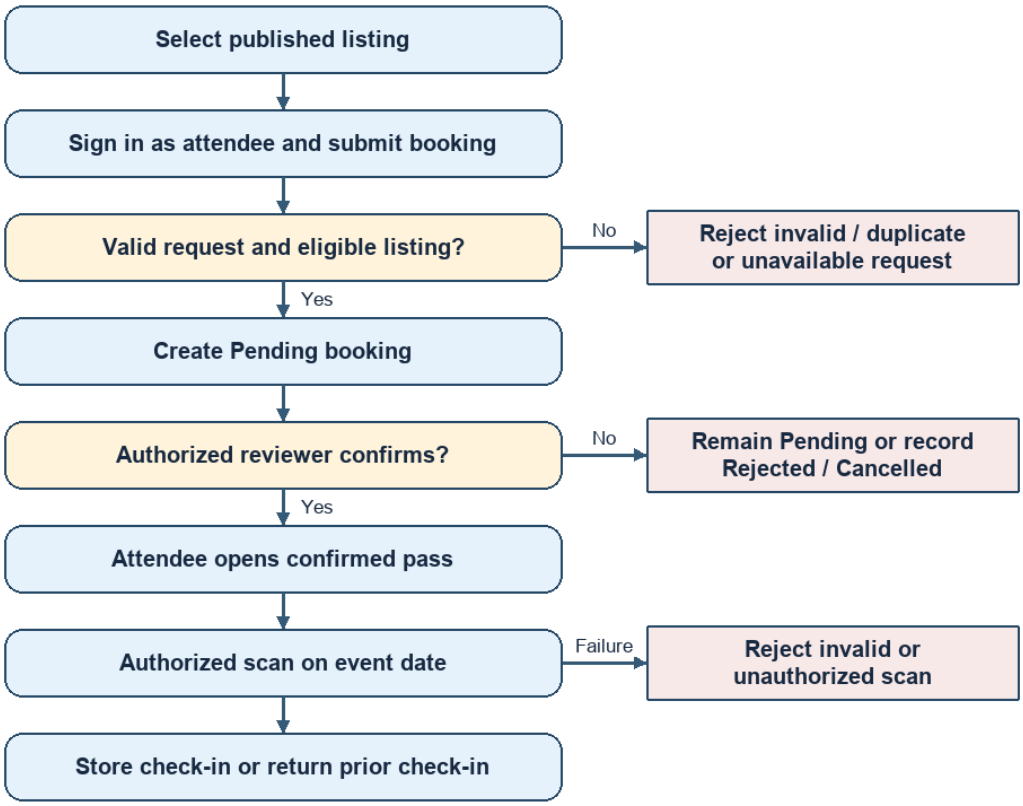


Figure 4.4. Attendee booking, approval, pass and check-in workflow.

The public verification page can confirm the pass status. Recording check-in is a separate, authenticated operation. A repeated valid scan returns the recorded check-in.

4.3 UML Sequence Diagram

The diagram follows a booth reservation request across the browser, authorization middleware, controller and database. The user must already have an approved application for the booth's listing.

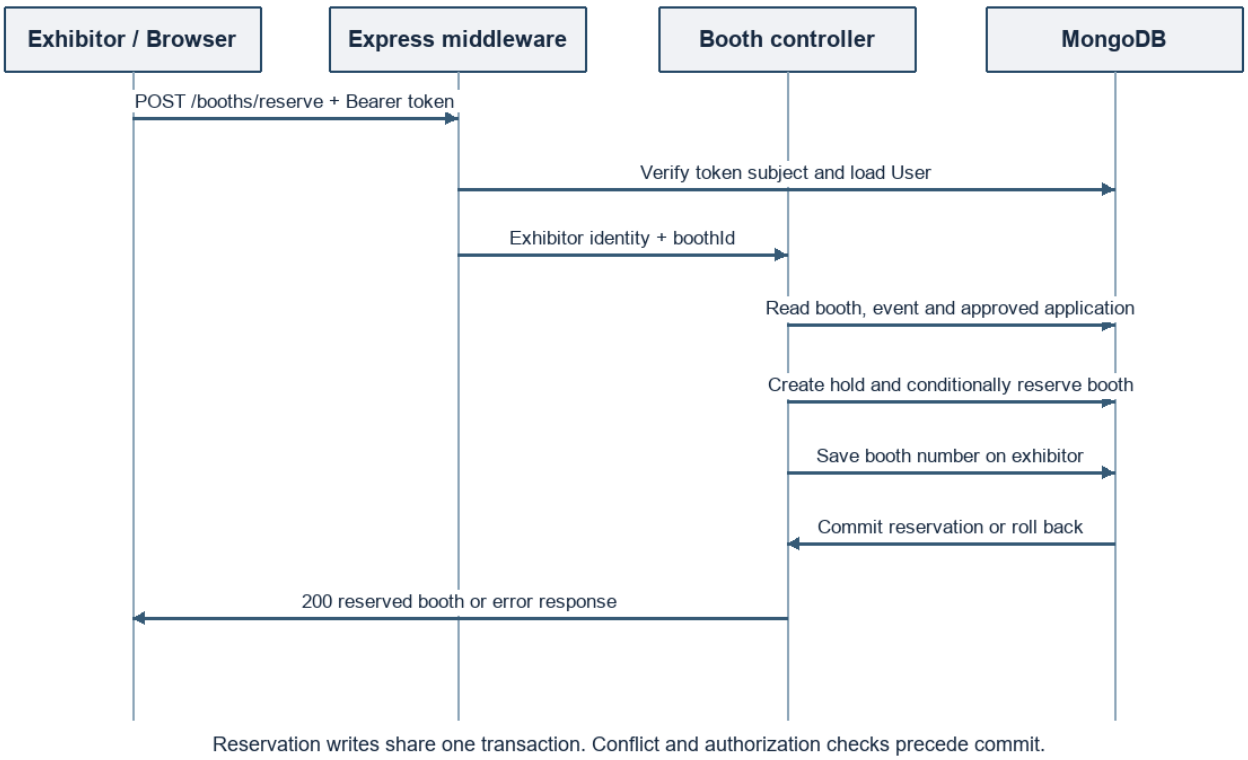


Figure 4.5. Sequence of calls and responses for POST /booths/reserve.

Authentication and role checks run before the transaction. Database writes share the transaction session. A duplicate hold or unavailable booth produces a conflict response.

4.4 Database Design

MongoDB stores operational entities as documents. Mongoose ObjectId references link accounts, listings, applications, booths and participation records. Floor plan placements and exhibitor staff members are embedded arrays.

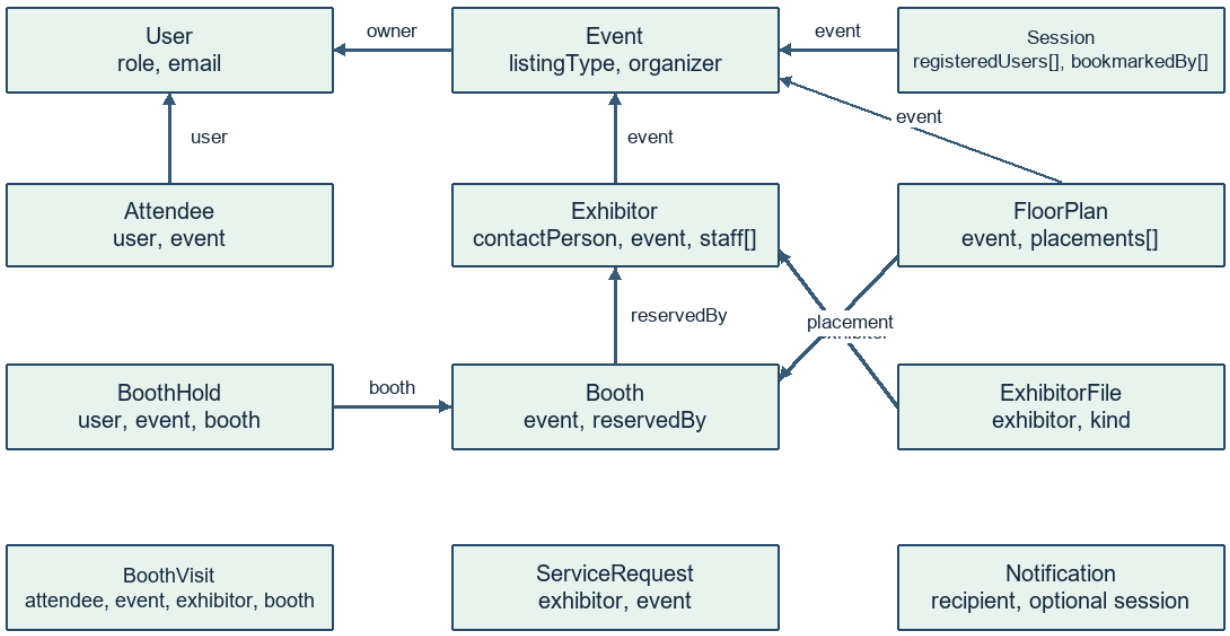


Figure 4.6. Principal MongoDB models and selected ObjectId references.

The following data dictionary gives the remaining relationships and key field constraints. In the diagram, the Event–Session link denotes the event reference stored on Session.

4.4.1 Operational Data Dictionary

Model	Principal fields and links	Key constraints
User	name, email, password, role, companyName, tokenVersion and reset fields	Unique email. Four role values. Sensitive fields are excluded from ordinary output.
Event	title, date, time, location, category, listingType, organizer, reviewExhibitor, reviewedBy, approvalStatus, coverData	Organizer and review references target User. Optimistic concurrency is enabled.
Attendee	user, event, ticketType, status, phone, guestCount, details, checkedInAt, checkedInBy	Unique user plus event. API creates Pending records.
Exhibitor	companyName, profile fields, contactPerson, event, boothNumber, status, staff[]	Unique contactPerson plus event. Up to 20 embedded staff entries.
Booth	event, boothNumber, status, reservedBy	Unique event plus boothNumber. Partial unique index on reservedBy.
BoothHold	user, event, booth	Partial unique index on user plus event.
FloorPlan	event, rows, columns, placements[].booth, row, column, revision	Unique event. Grid dimensions from 1 to 20. Up to 400 placements.
BoothVisit	attendee, event, exhibitor, booth	Unique attendee plus exhibitor. Event and exhibitor support lookup.

The Attendee schema’s default status is Confirmed, while the registration controller explicitly writes Pending for newly submitted bookings. The unique index applies to the user and listing regardless of status.

4.4.2 Supporting Data Dictionary

Model	Principal fields and links	Key constraints
Session	event, title, speaker, topic, location, startTime, endTime, registeredUsers[], bookmarkedBy[]	User references in both activity arrays. Arrays excluded from default query selection.
Notification	recipient, message, read, optional session, startTime, reminderKey	Recipient references User. Sparse unique reminder key.
Message	event, participant, sender, text	Participant and sender reference User. Thread queries use the event and participant.
Feedback	user, optional event, subject, rating, type, status, reply, message	Type is suggestion, issue or review. Status is Open, In Progress or Resolved.
ExhibitorFile	exhibitor, kind, filename, contentType, size, data	Unique exhibitor plus kind. Supported kinds are logo, photo and document.
ServiceRequest	exhibitor, event, service, details, status, reply	Service and status use enumerated values. Indexed by exhibitor and by event plus status.
WebsiteBranding	String _id, data, filename, version	Website logo uses the singleton ID website.

The repository also includes forgetmodel.js, which declares an email and OTP schema. The active recovery controller writes reset-token fields on User. The OTP schema does not participate in that flow.

Pass generation derives a signed token from the existing Attendee booking ID. The PDF and QR are generated when requested, and check-in timestamps are written to the booking.

5. Package Dependencies and Technical Justification

5.1 Backend Package Inventory

The tables use manifest version ranges. The lockfiles record resolved dependencies for reproducible installation. Tier denotes the manifest section.

Backend package	Range	Tier	Purpose
bcrypt	^6.0.0	Runtime	Password hashing and comparison.
cors	^2.8.6	Runtime	Cross-origin browser access.
dotenv	^17.4.2	Runtime	Environment configuration.
express	^5.2.1	Runtime	HTTP routing and middleware.
express-rate-limit	^8.7.1	Runtime	Rate limits for sensitive endpoints.
jsonwebtoken	^9.0.3	Runtime	Login and booking-pass signatures.
mongoose	^9.9.5	Runtime	Document models, indexes and transactions.
multer	^2.5.0	Runtime	Multipart upload processing.
nodemailer	^10.0.10	Runtime	SMTP delivery for recovery.
nodemon	^3.1.14	Runtime	Development server restart.
otp-generator	^4.0.1	Runtime	Declared OTP utility. Current recovery stores reset fields on User.
pdf-lib	^1.17.1	Runtime	Booking-pass PDF rendering.
qrcode	^1.5.4	Runtime	QR generation for pass links.
sharp	^0.35.5	Runtime	Image conversion and resizing.
oxlint	^1.91.0	Dev.	Source linting.

5.2 Frontend Package Inventory

Frontend package	Range	Tier	Purpose
@animateicons/react	^0.13.0	Runtime	Animated workspace icons.
@dnd-kit/core	^6.3.1	Runtime	Drag-and-drop primitives.
@dnd-kit/modifiers	^9.0.0	Runtime	Drag behavior modifiers.
@dnd-kit/sortable	^10.0.0	Runtime	Sortable interactions.
@dnd-kit/utilities	^3.2.2	Runtime	Drag-and-drop utilities.
@fontsource-variable/geist	^5.2.9	Runtime	Bundled variable font.
@reduxjs/toolkit	^2.13.0	Runtime	Redux store and async state actions.
@tailwindcss/vite	^4.3.1	Runtime	Tailwind integration with Vite.
@tanstack/react-table	^8.21.3	Runtime	Headless table state and row models.
@vercel/analytics	^2.0.1	Runtime	Vercel analytics integration.
axios	^1.18.1	Runtime	HTTP client and interceptors.
class-variance-authority	^0.7.1	Runtime	Variant-driven class composition.
clsx	^2.1.1	Runtime	Conditional class composition.
lucide-react	^1.23.0	Runtime	Interface icons.
next-themes	^0.4.6	Runtime	React theme utilities.
radix-ui	^1.6.0	Runtime	UI primitives.
react	^19.2.6	Runtime	Component and rendering model.
react-dom	^19.2.6	Runtime	Browser mounting.

5.2 Frontend Package Inventory Continued

Frontend package	Range	Tier	Purpose
react-redux	^9.3.0	Runtime	Connects React with Redux.
react-router-dom	^7.18.1	Runtime	Client routing and route guards.
react-toastify	^11.1.0	Runtime	User feedback toasts.
recharts	^3.8.0	Runtime	Operational charts.
shadcn	^4.11.0	Runtime	UI component tooling.
sonner	^2.0.7	Runtime	Toast components.
tailwind-merge	^3.6.0	Runtime	Resolves Tailwind utility conflicts.
tailwindcss	^4.3.1	Runtime	Utility styling.
tw-animate-css	^1.4.0	Runtime	Animation utilities.
vaul	^1.1.2	Runtime	Drawer primitive.
zod	^4.4.3	Runtime	Schema validation utility.
@types/react	^19.2.14	Dev.	React tooling types.
@types/react-dom	^19.2.3	Dev.	React DOM tooling types.
@vitejs/plugin-react	^6.0.1	Dev.	Vite integration for React.
globals	^17.6.0	Dev.	Global identifiers for tooling.
oxlint	^1.91.0	Dev.	Source linting.
vite	^8.0.12	Dev.	Development server and bundler.

Development tools may be listed under runtime dependencies. For example, nodemon is used by the backend development command.

6. System Folder Structure

The repository is organized into separate frontend and backend applications. Each application has its own package manifest and lockfile.

```
E-Project/
|-- README.md
|-- Backend/
|   |-- index.js                    Database and server startup
|   |-- package.json / package-lock.json
|   |-- scripts/                    Backend source checks
|   |-- tests/                      Node regression tests
|   |-- src/
|       |-- app.js                  Express configuration and route mounts
|       |-- configs/                Database and mail settings
|       |-- controllers/            Request handling and domain operations
|       |-- middleware/             Identity and role checks
|       |-- models/                 Mongoose schemas and indexes
|       |-- presentation/           PDF booking-pass layout
|       |-- routes/                 Paths and access requirements
|       |-- services/               Workflow rules, events and reminders
|-- Frontend/
    |-- index.html / vite.config.js / vercel.json
    |-- package.json / package-lock.json
    |-- public/                     Site icons
    |-- src/
        |-- main.jsx / App.jsx      Browser entry and routes
        |-- api/                    Shared Axios client
        |-- assets/                 Website imagery
        |-- components/             Public, auth, workspace and UI elements
        |-- context/                Theme and sidebar context
        |-- hooks/                  Shared data and application hooks
        |-- layout/                 Authenticated workspace shell
        |-- lib/                    Validation, date and style helpers
        |-- page/                   Public pages and workspace screens
        |-- redux/                  Store and authentication slice
```

Request and Rendering Entry Points

Frontend/src/main.jsx mounts the browser app and App.jsx selects the route. Backend/index.js connects to MongoDB and starts the API. Backend/src/app.js mounts the routers, which call controllers and services to read or change records.

7. Screenshots and Interface Walkthrough

The figures show the built EventSphere interface in a desktop browser. The walkthrough uses demonstration accounts and sample records supplied through controlled API responses.

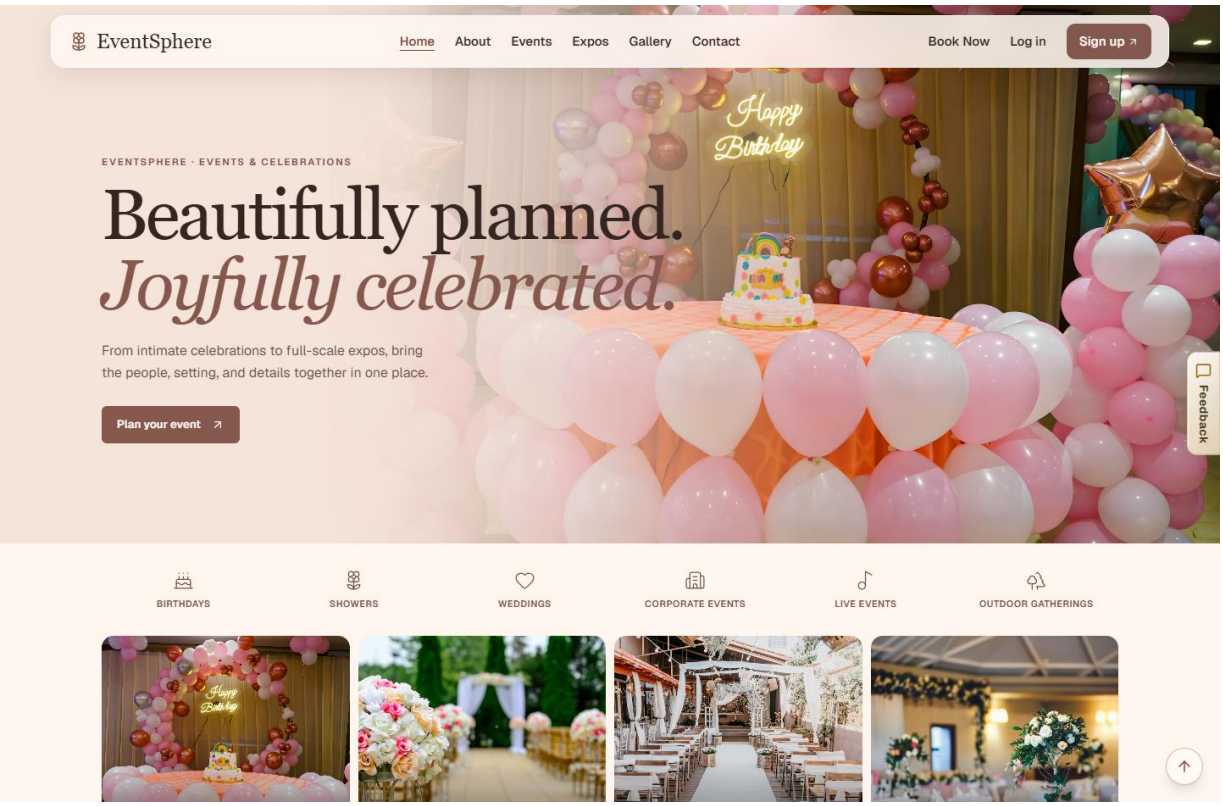


Figure 7.1. Public landing page with navigation and booking entry point.

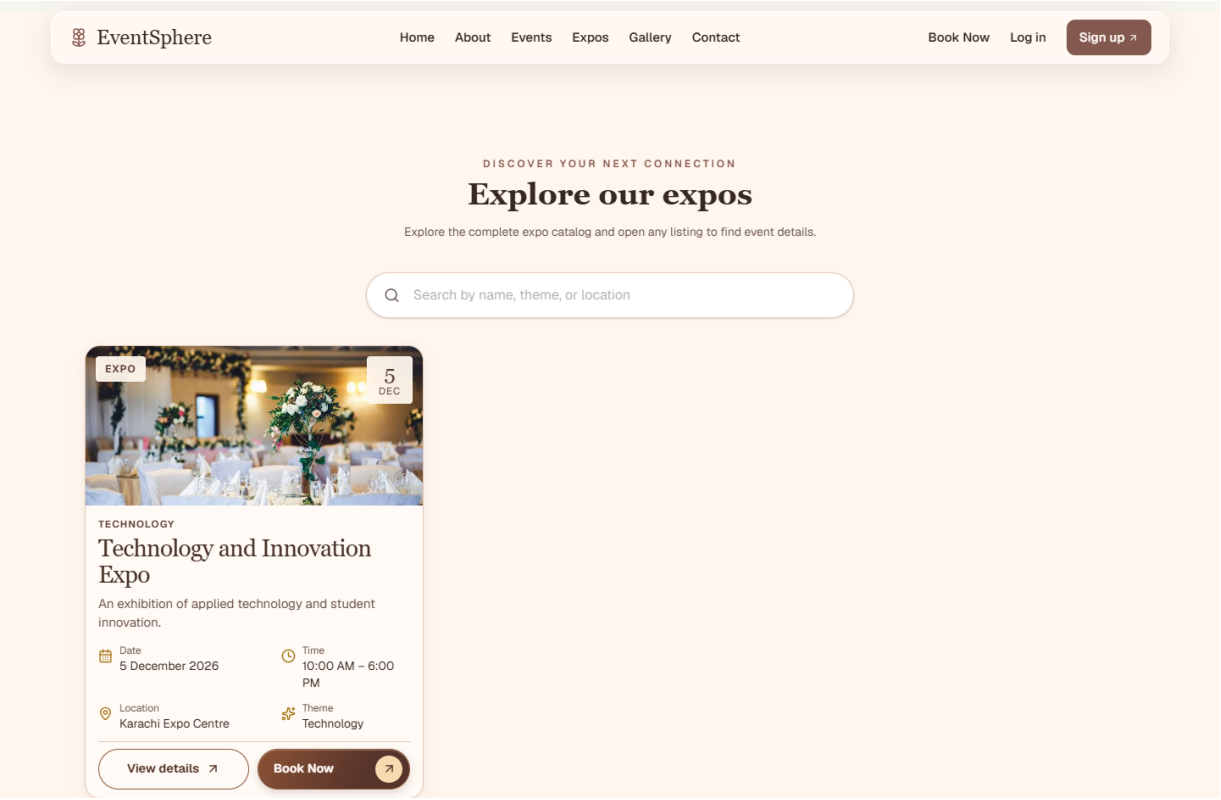


Figure 7.2. Public Expo listing with search and a sample listing card.

7.2 Authentication Pages

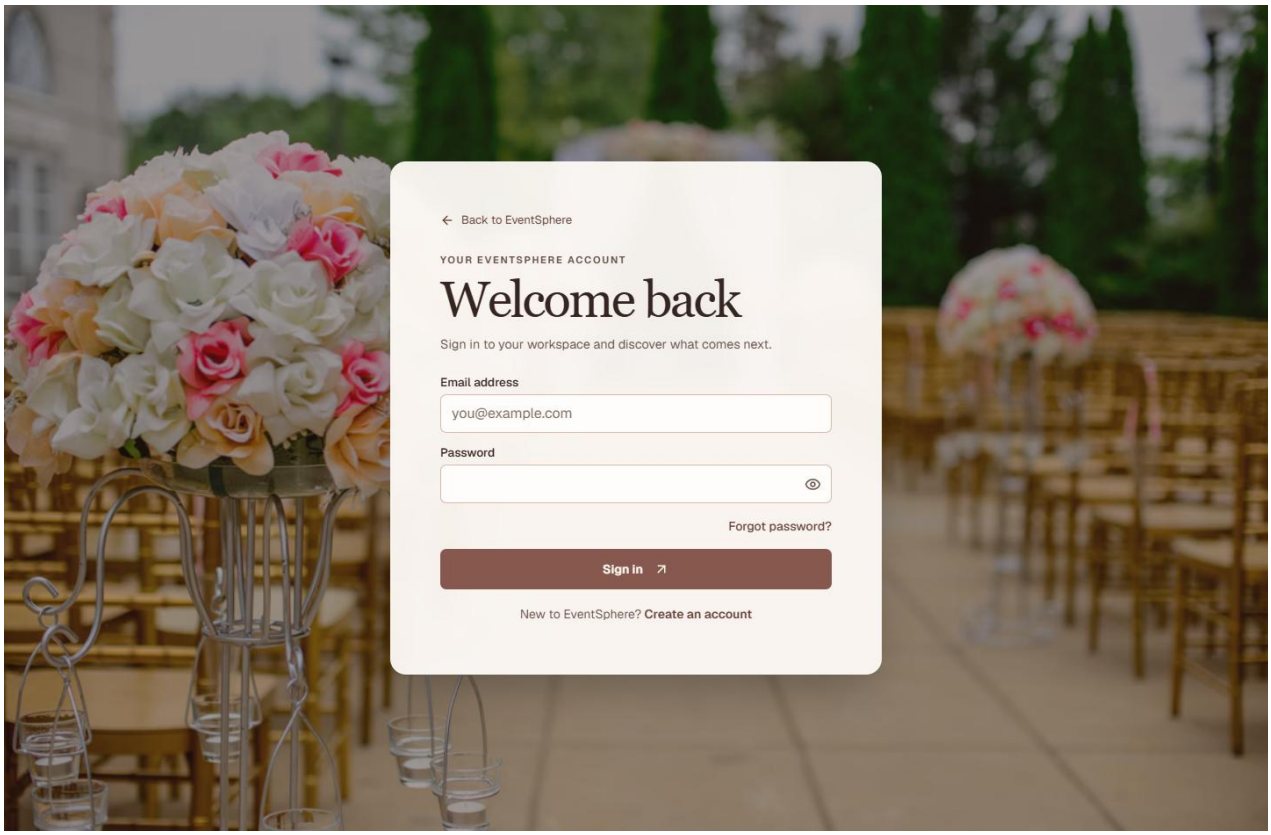


Figure 7.3. Login page with credentials and password recovery entry.

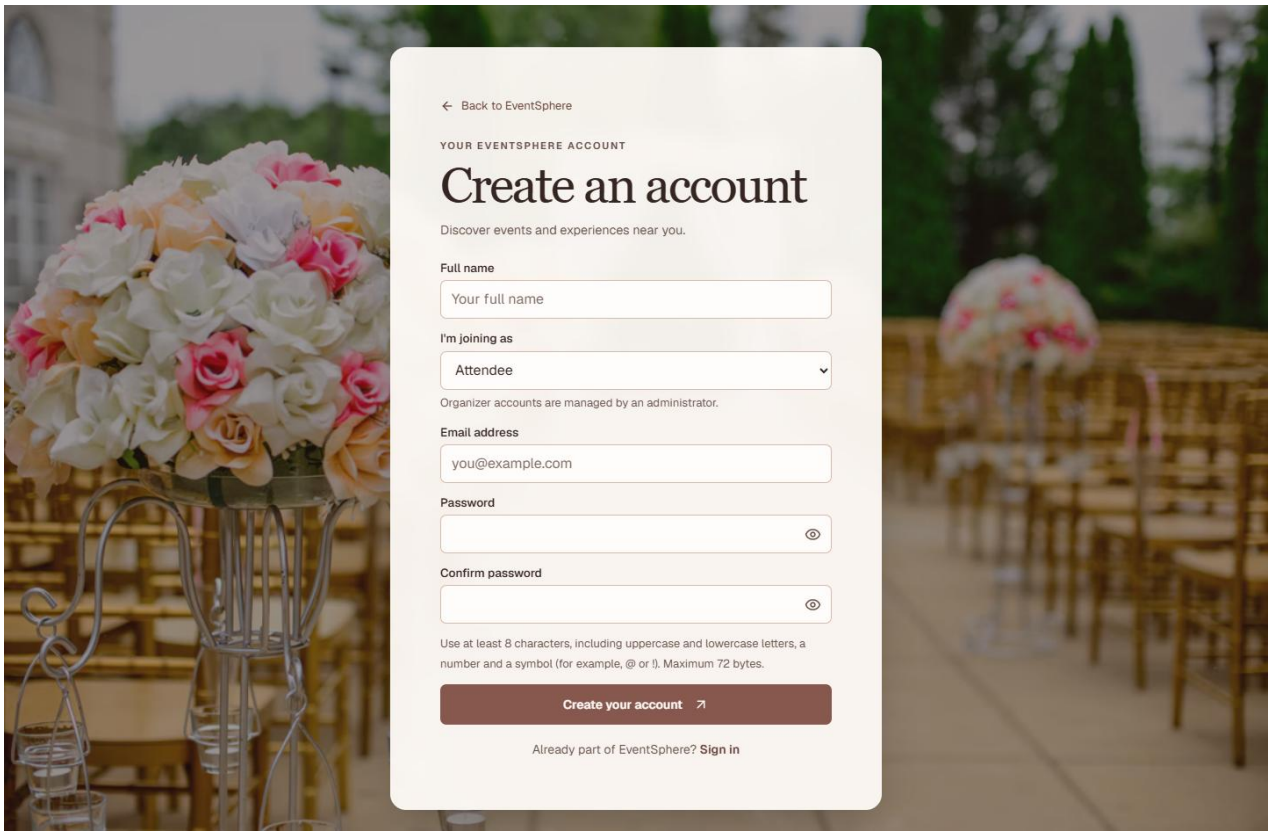


Figure 7.4. Account creation page with role selection and password guidance.

7.3 Administrator Workspace

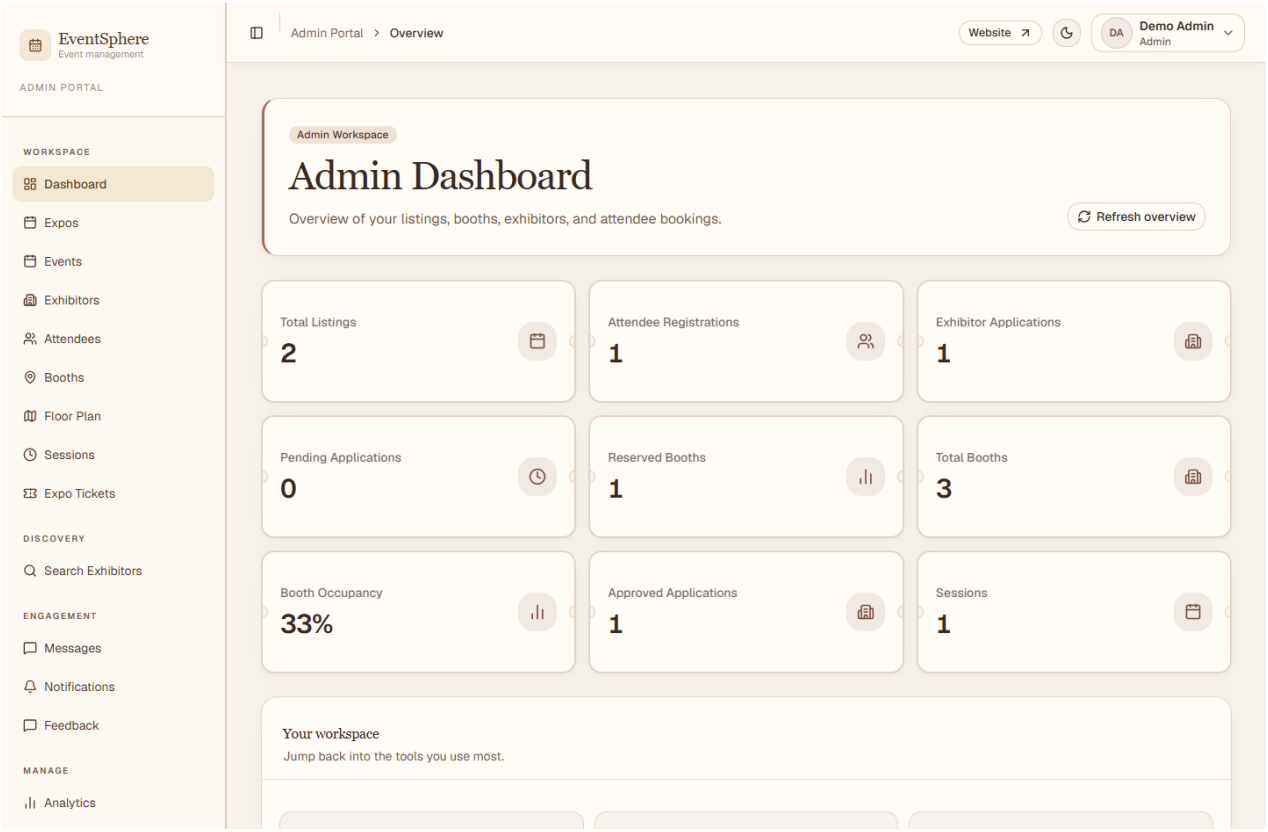


Figure 7.5. Administrator dashboard and workspace navigation.

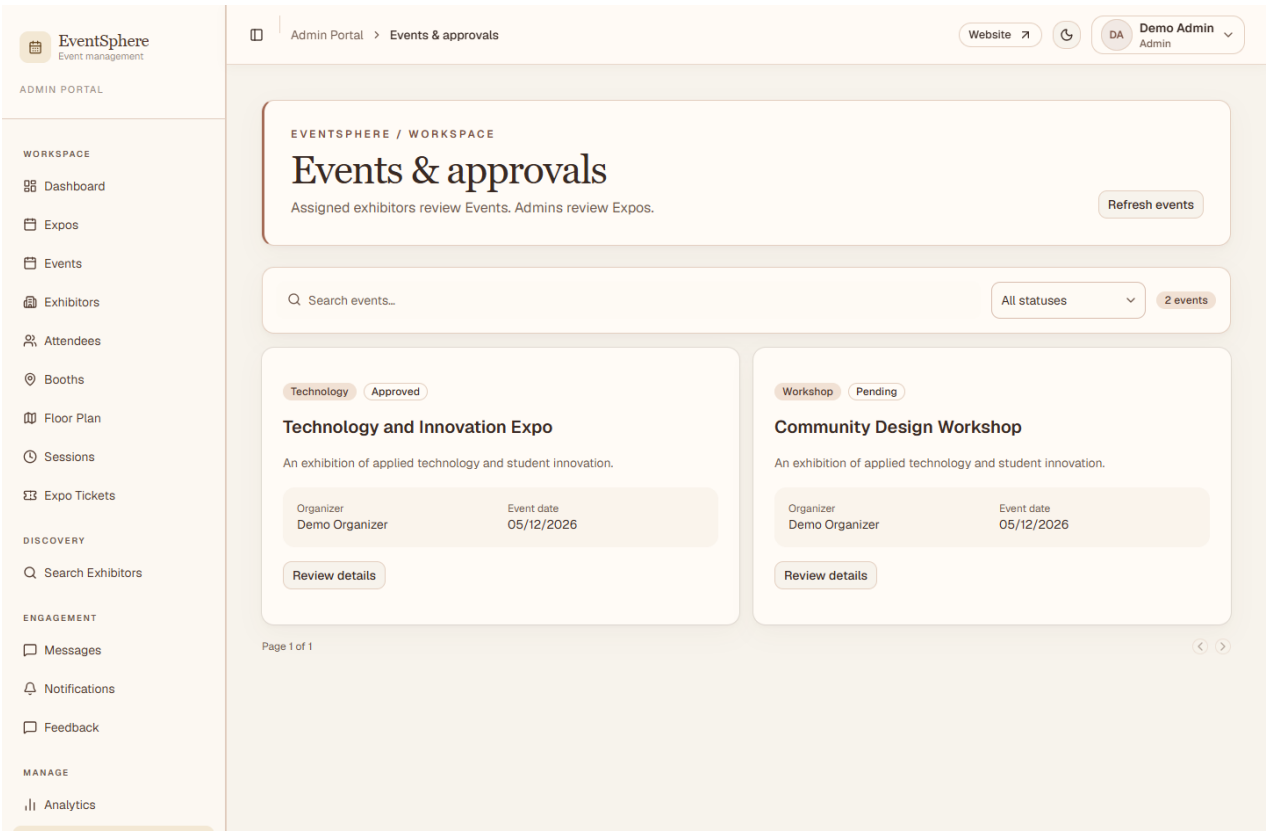


Figure 7.6. Events and Approvals with an approved Expo and a pending Event.

7.4 Expo Operations

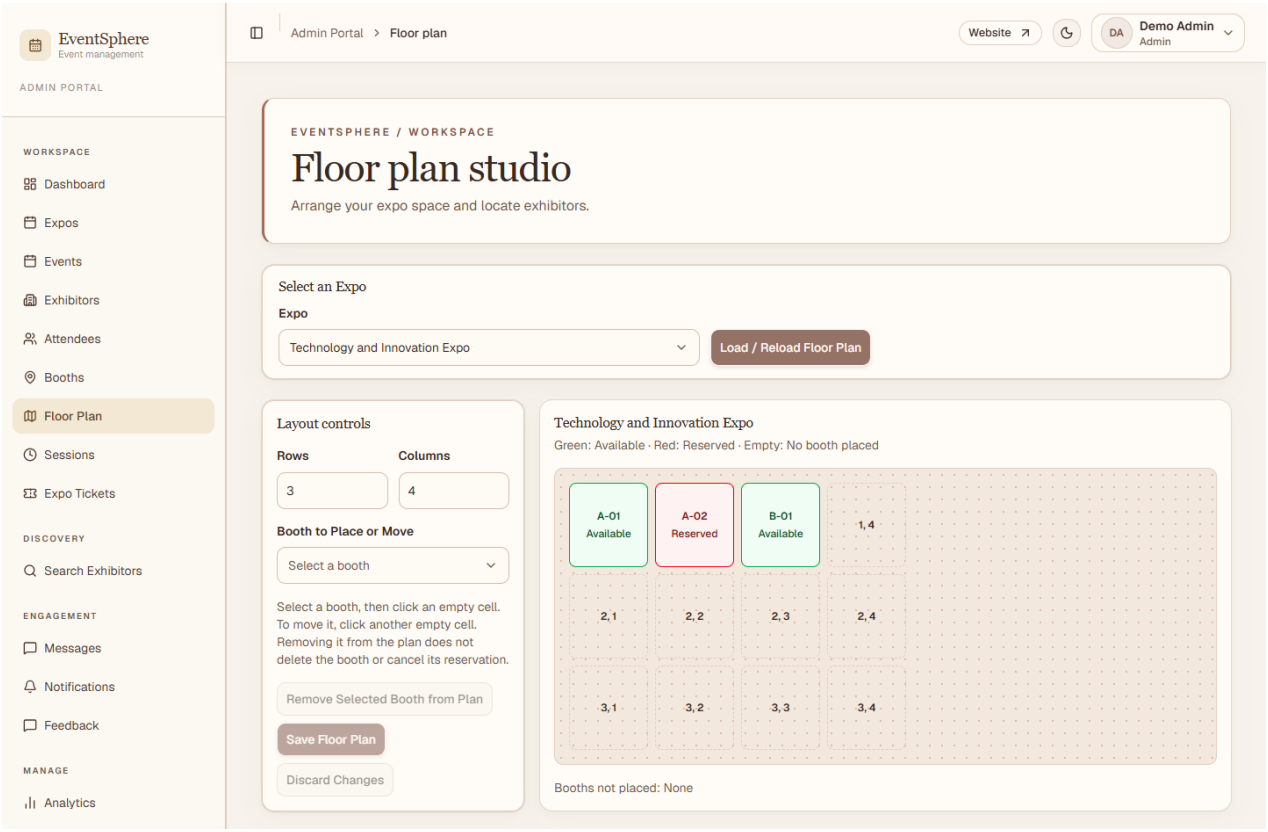


Figure 7.7. Floor Plan Studio and Expo booth layout.

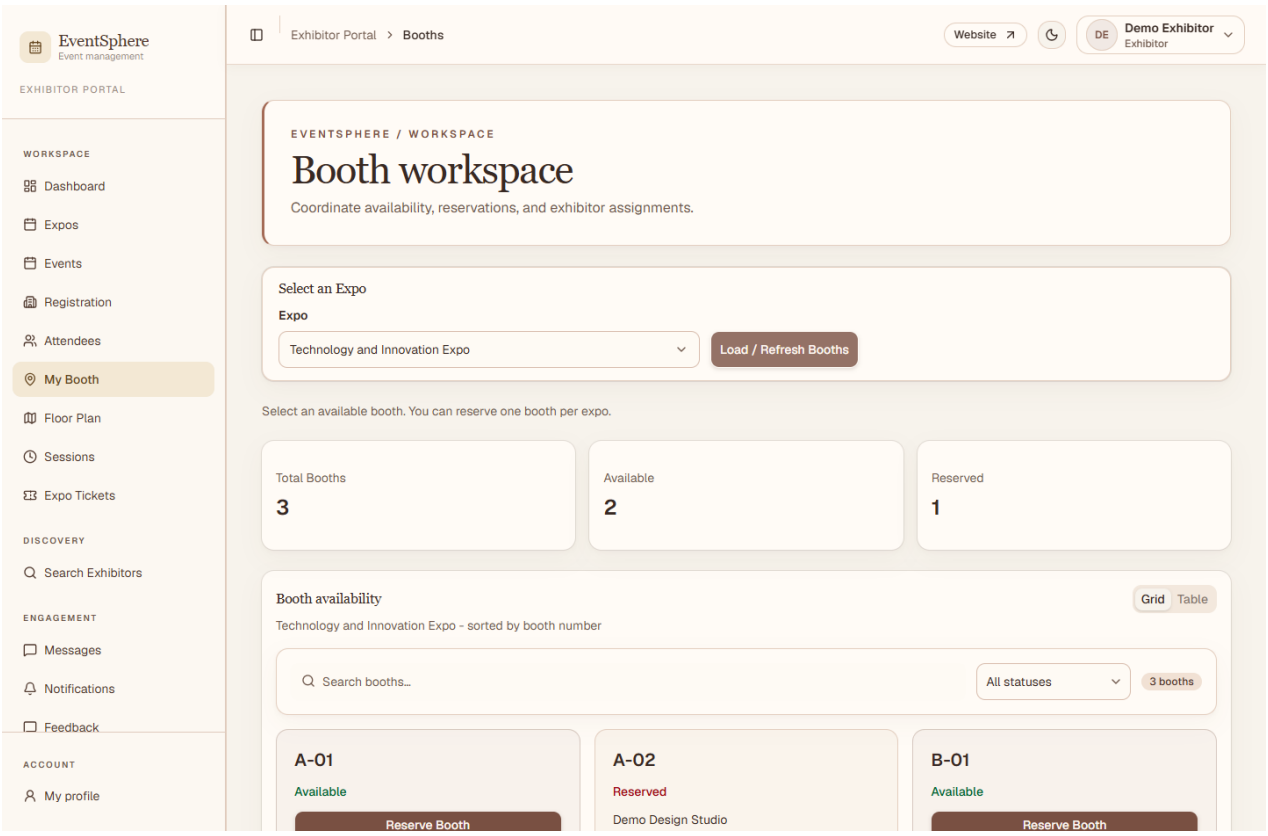


Figure 7.8. Exhibitor booth workspace for viewing availability.

7.5 Exhibitor and Attendee Dashboards

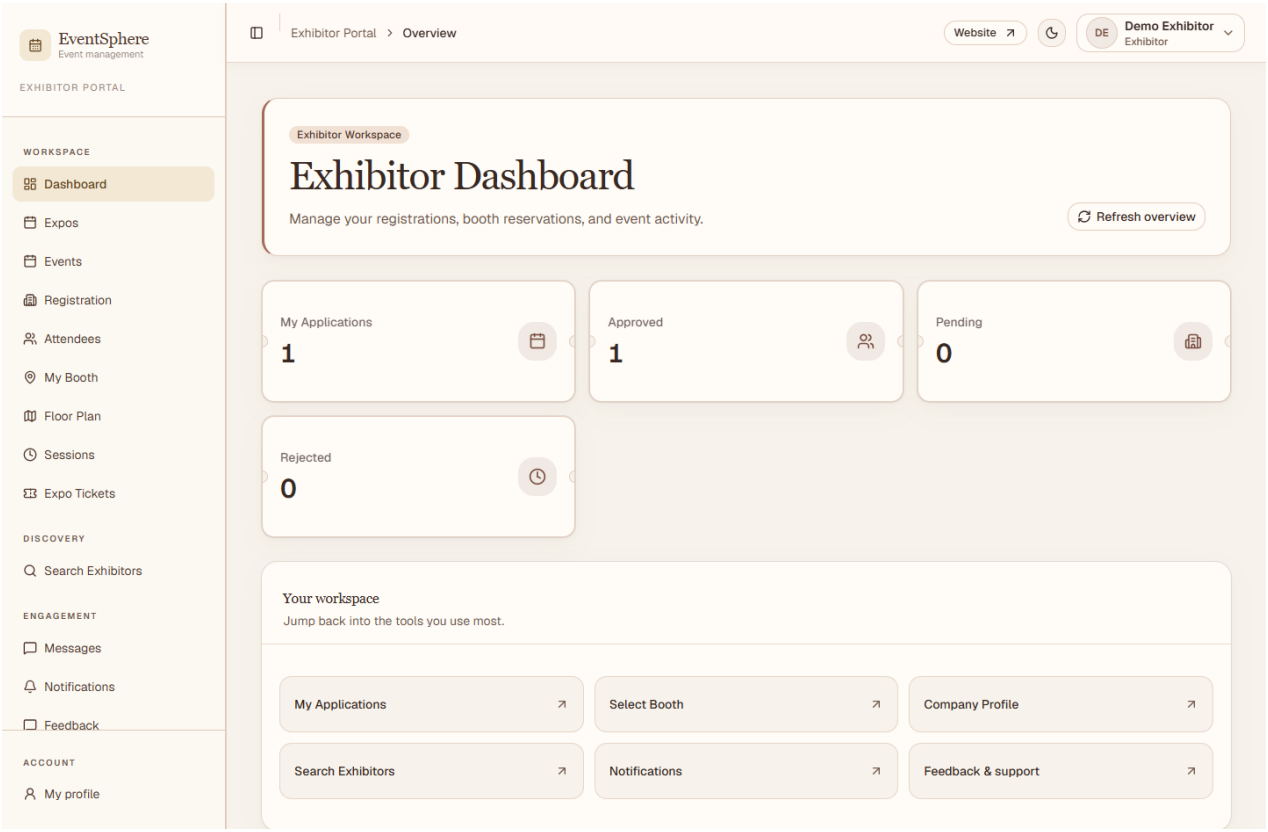


Figure 7.9. Exhibitor dashboard with application and booth summaries.

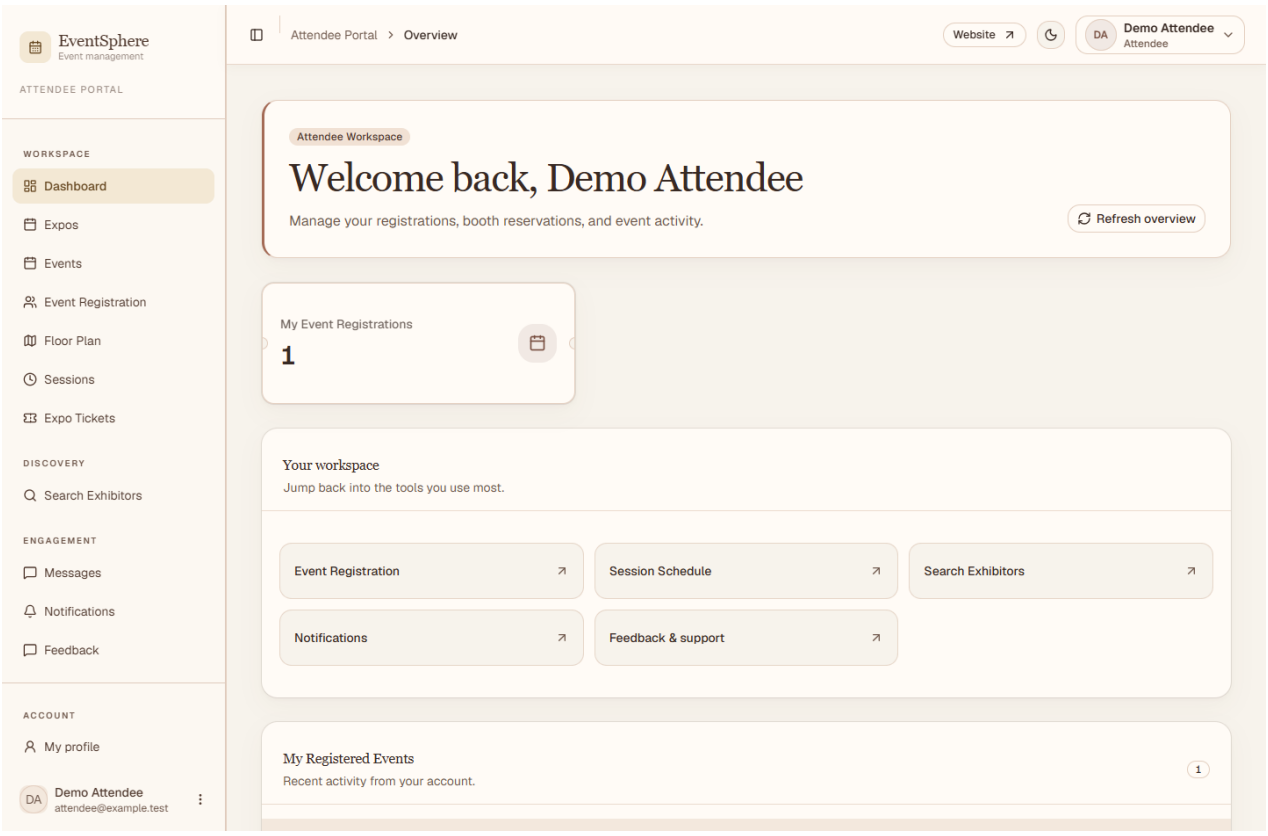


Figure 7.10. Attendee dashboard with registration summary and navigation.

7.6 Ticket Management and Attendee Profile

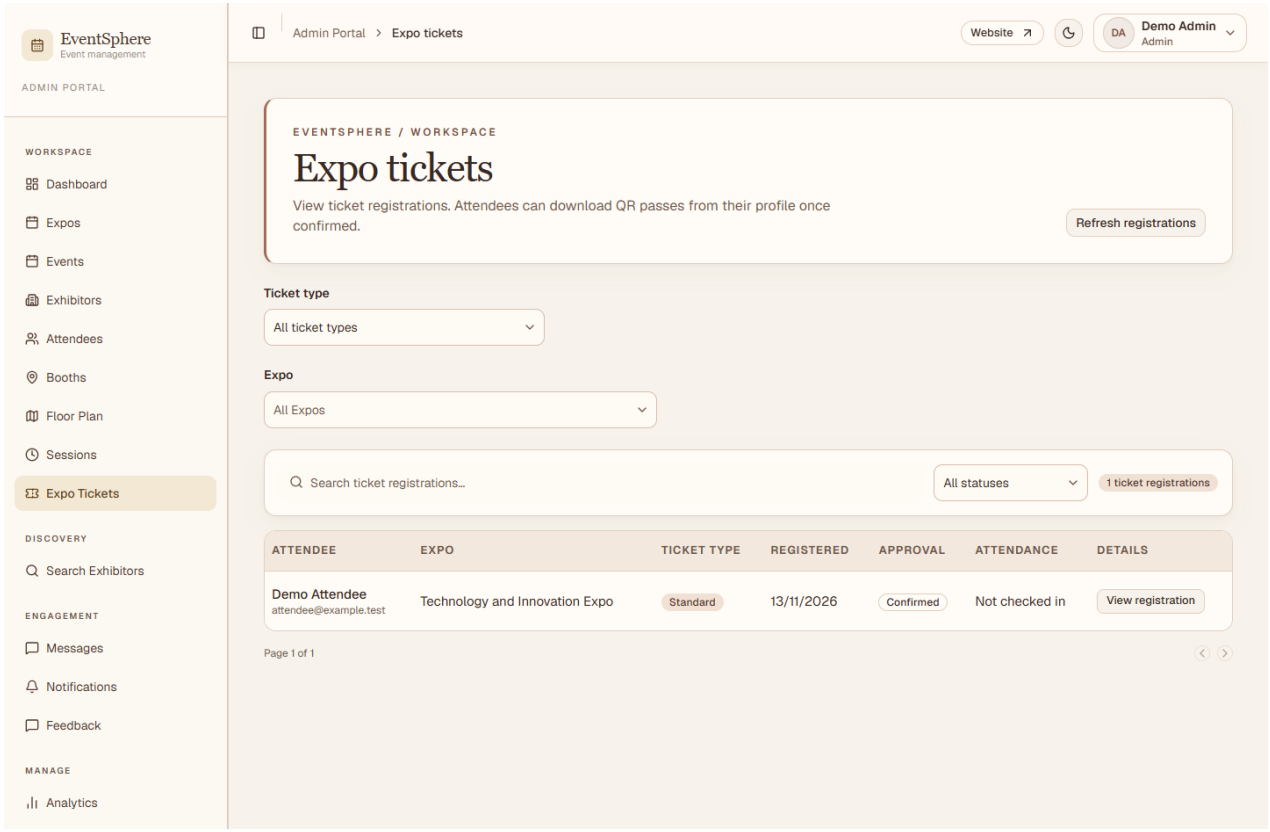


Figure 7.11. Expo Tickets workspace with booking status and check-in access.

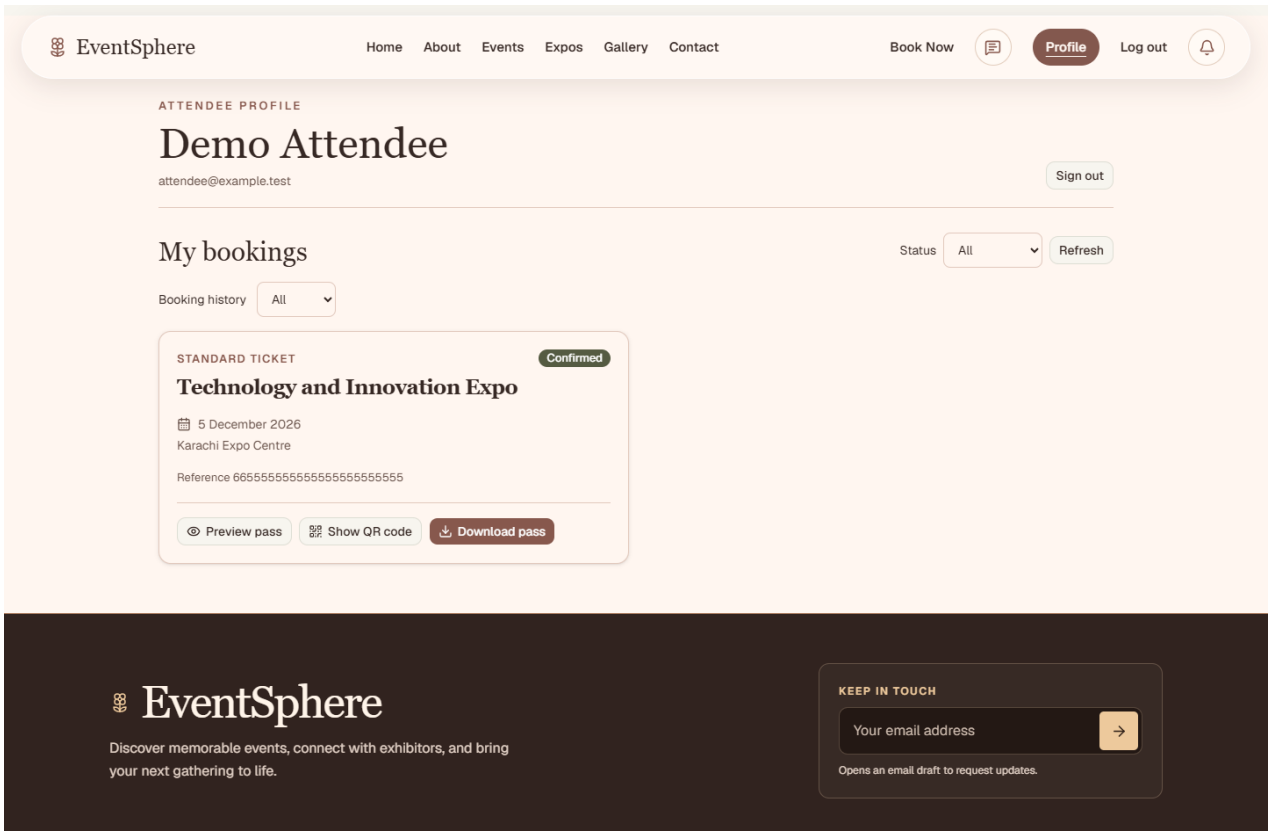


Figure 7.12. Attendee profile showing a confirmed booking and pass actions.

8. Source Code with Comments

8.1 Bearer Token Validation

The middleware restricts JWT verification to HS256 and checks the issuer and audience. It then loads the User and compares token versions, which invalidates an old session after password reset.

Backend/src/middleware/authmiddleware.js · lines 38–55

```
  try {
    payload = jwt.verify(match[1], secret, {
      algorithms: ["HS256"],
      issuer: "eventsphere-api",
      audience: "eventsphere-web",
    });

    if (
      typeof payload !== "object" ||
      !mongoose.isObjectIdOrHexString(payload.sub) ||
      !Number.isInteger(payload.version)
    ) {
      return res.status(401).json({
        msg: "Please log in again",
      });
    }
  } catch {
    return res.status(401).json({
```

8.2 Client Authorization Header

The Axios interceptor attaches the current bearer token. The response interceptor clears a rejected session only when the failed request used the currently stored token.

Frontend/src/api/api.js · lines 34–42

```
api.interceptors.request.use((config) => {
  const token = localStorage.getItem("token");

  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }

  return config;
});
```

8.3 Transactional Booth Reservation

The controller creates a hold before performing a conditional booth update. The match requires Available status and an unassigned reservedBy field. If the write fails, the error exits the transaction. On success, the application booth number is saved in the same transaction.

Backend/src/controllers/boothcontroller.js · lines 132–159

```
await claimBooth(req.user._id, boothId, booth.event, session);
const updated = await Booth.findOneAndUpdate(
  {
    _id: boothId,
    status: "Available",
    reservedBy: null,
  },
  {
    $set: {
      status: "Reserved",
      reservedBy: exhibitor._id,
    },
  },
  {
    returnDocument: "after",
    runValidators: true,
    session,
  }
);

if (!updated) {
  fail(409, "This booth is already reserved");
}

exhibitor.boothNumber = updated.boothNumber;
await exhibitor.save({ session });

return updated;
```

Concurrency safeguards

A unique hold limits a user to one reservation per event. The partial unique reservedBy index prevents more than one booth from pointing to the same application.

8.4 Idempotent Booking Check-In

The controller verifies the signed booking token, the reviewer's authority, the booking status and the event date in Asia/Karachi. It conditionally records the operator and time on a booking without an existing check-in.

Backend/src/controllers/bookingpasscontroller.js · lines 62–73

```
  const updated = await Attendee.findOneAndUpdate(
    { _id: id, $or: [{ status: "Confirmed" }, { status: { $exists: false } }], checkedInAt: null },
    { $set: { checkedInAt: new Date(), checkedInBy: req.user._id } },
    { new: true },
  );
  if (!updated) {
    const latest = await Attendee.findById(id);
    if (latest?.status !== "Confirmed") return res.status(409).json({ msg: "This ticket is no longer
confirmed." });
    return res.json({ title: booking.event.title, checkedInAt: latest.checkedInAt, alreadyCheckedIn:
true });
  }
  return res.json({ title: booking.event.title, checkedInAt: updated.checkedInAt, alreadyCheckedIn:
false });
}
```

Repeated scans

If the conditional write does not return a document, the controller rereads the booking. A still-confirmed booking returns the existing check-in time and indicates that it was already checked in.

9. User Guide

9.1 Getting Started

1. Open the website. In local development, use the address shown by Vite, usually http://localhost:5173.

2. Browse Events or Expos and open a listing.

3. Sign in or choose Attendee or Exhibitor on the signup page. An exhibitor account requires a company name.

4. For account recovery, open the password recovery link from the login page, submit the account email and follow the emailed link.

9.2 Attendee Workflow

1. Choose an approved Event or Expo and open its booking form. Sign in as an Attendee if prompted.

2. Complete the form and submit the request. Open My Profile or the registrations workspace to check its status.

3. When the booking becomes Confirmed, preview or download the pass. Present the QR at entry.

4. Use Sessions to register for or bookmark a session. Open Notifications to read reminders.

5. Use Floor Plan and Search Exhibitors to explore an Expo. Record an eligible booth visit from the workspace.

9.3 Administrator and Organizer Workflows

Listing Management

1. Open Expos or Events. Create or edit a listing according to role. A regular Event requires an assigned exhibitor reviewer.

2. Open Events & Approvals. An Administrator reviews an Expo. The assigned exhibitor reviews a regular Event. Reload before retrying a stale review.

3. Use Exhibitors and Booths to review applications and manage assignments. In Floor Plan, select the Expo, load the layout, place booths and save.

4. Use Sessions to manage the schedule and Analytics to inspect scoped counts.

Booking Review and Entry

1. As Administrator, open Expo Tickets. As assigned Exhibitor, open the relevant attendee registration workspace for a regular Event.

2. Review the Pending booking and select an available status action. A Confirmed booking can generate a pass.

3. On the event date, open the scanner and scan the QR or enter its data manually when available. Review the check-in result.

Administration

Administrators can list accounts in Users and update the shared logo in Website Settings. Banner settings apply to the current browser. Organizers manage resources within their permitted listing scope.

9.4 Exhibitor Workflow

1. Register as an Exhibitor with a company name or sign in.

2. Open Exhibitors and apply for an eligible Expo. Wait for application approval before requesting a booth.

3. Open Booths, select the Expo and load availability. Reserve an available booth or cancel an existing reservation when permitted.

4. Use the exhibitor profile and application controls to update details, upload files, maintain staff and request services such as Internet or furniture.

5. Use Messages for eligible listing threads. If assigned to a regular Event, review the listing and its attendee requests in the appropriate workspace.

9.5 Communication and Support

Use Messages for permitted listing conversations and Feedback for suggestions, issues or reviews. The public Contact form prepares a message in the visitor’s email application.

10. Developer Guide and Module Descriptions

10.1 Architecture and Technology Rationale

React renders public and protected pages, React Router selects screens, and Axios sends bearer credentials. Express delegates request handling to controllers and services, and Mongoose maps the underlying MongoDB documents. Server-sent change notifications prompt selected views to refresh.

10.2 Local Development Setup

1. Install Node.js and npm. Provision a development MongoDB database that supports transactions.

2. Install both applications and configure the variables below in Backend/.env and, as needed, Frontend/.env.

3. Start the API and frontend in separate terminals. Verify database readiness with GET /health.

```
npm --prefix Backend ci
npm --prefix Frontend ci
npm --prefix Backend run dev
npm --prefix Frontend run dev
```

Variable	Location and use
MONGODB_URL	Backend. Connection URI for the database.
JWT_SECRET	Backend. Strong random secret of at least 32 characters.
PORT	Backend. Defaults to 3000.
FRONTEND_URL	Backend. Website origin used in recovery links.
EMAIL_USER / EMAIL_PASS	Backend. Mail transport credentials.
PUBLIC_WEBSITE_URL	Backend. Public HTTPS frontend origin for QR links.
VITE_API_URL	Frontend. API origin, set at build time. Defaults to http://localhost:3000.

Use authorized evaluation accounts for admin and organizer access. Public signup creates attendee and exhibitor accounts.

10.3 API Route Reference

Prefixes match Backend/src/app.js. Protected routes require a Bearer token. Controllers also enforce ownership and workflow rules.

Mounted prefix and module	Methods and relative paths
/auth Registration, login, identity and Admin account listing.	POST /login POST /register GET /me GET /allusers
/forget Recovery email and password reset.	POST /verifyemail POST /resetPassword
/events Public listings, scoped workspace, listing review and media.	GET /all GET /workspace GET /reviewers PATCH /:id/review GET /:id/image GET /:id POST /create PUT /update/:id DELETE /delete/:id
/website-settings Read, replace or remove the shared website logo.	GET /logo GET /logo/image PUT /logo DELETE /logo
/attendees Bookings, approval, pass issuance, verification and check-in.	POST /passes/verify POST /passes/check-in GET /:id/pass PATCH /:id/status POST /register GET /all

GET /health reports readiness when database middleware succeeds. GET /updates streams change signals.

10.3 API Route Reference Continued

Mounted prefix and module	Methods and relative paths
/booths Availability, layout, visits and allocation.	GET /event/:eventId GET /layout/:eventId PUT /layout/:eventId GET /visits/mine POST /visit POST /create POST /reserve POST /cancel POST /assign
/exhibitors Applications, public search, profiles, staff and service requests.	GET /search POST /register GET /all GET /staff/:id PUT /staff/:id GET /services/:id POST /services/:id PUT /service-requests/:requestId/status PUT /profile/:id PUT /status/:id
/exhibitor-files Public logo and controlled media upload or retrieval.	GET /:id/logo GET /:id GET /:id/private-logo GET /:id/document GET /:id/private-photo POST /:id/:kind

Booth reservation, assignment and cancellation use transactions. The floor plan controller checks the layout revision. Exhibitor media routes enforce visibility by file kind and requester.

10.3 API Route Reference Continued

Mounted prefix and module	Methods and relative paths
/sessions Public schedule, attendee activity and staff management.	GET /event/:eventId GET /:id/activity PUT /:id/activity POST /create PUT /update/:id DELETE /delete/:id
/notifications Current user's notifications and read state.	GET /mine PATCH /read-all PATCH /:id/read
/analytics Scoped operational summary.	GET /summary
/feedback Public reviews, submissions and staff issue handling.	GET /reviews POST /submit GET /all PUT /update/:id
/messages Eligible listing conversations and messages.	GET /conversations GET /thread/:eventId/:participantId POST /thread/:eventId/:participantId

The backend checks upcoming sessions every 30 seconds, using a 15-minute look-ahead and a unique reminder key.

Module Entry Points

Module	Source files
Listing publication	eventcontroller.js, eventreviewcontroller.js and eventworkflow.js
Ticketing and entry	attendeecontroller.js, bookingpasscontroller.js and presentation/bookingpass.js
Booth and floor plan	boothcontroller.js, boothassignmentcontroller.js, boothholds.js and floorplancontroller.js
Sessions and reminders	sessionactivitycontroller.js and sessionreminders.js
Client synchronization	Frontend/src/components/LiveUpdates.jsx and hooks/useLiveRefresh.js

10.4 Verification and Troubleshooting

Development Checks

Check	Observed result
Backend syntax check	Completed successfully.
Backend test suite	11 passed. The tests cover listing and booking review, check-in, booth holds, pass links, signup and website branding.
Frontend production build	Completed successfully with build output directed to a temporary folder.
Frontend lint	Windows Application Control blocked the Oxlint native binding before analysis.

Common Operating Issues

- If the API reports database unavailability, verify the connection URI and network access.
- If transactional booth operations fail on a standalone server, configure a MongoDB replica set or compatible managed cluster.
- If the QR opens the wrong site, check PUBLIC_WEBSITE_URL and generate a fresh pass. The verification site needs the booking database and the corresponding signing secret.
- If the frontend cannot reach the API, verify VITE_API_URL and rebuild after changing it.
- If recovery email does not arrive, verify the configured website origin, mail settings and spam folder.

Repository References

Implementation references include the root README, both package manifests and lockfiles, Backend/README.md, Backend/src, Backend/tests and Frontend/src.